

# להבין את המערכת – מבט מאחורי הקלעים

בנינו את הכלים, התחלנו ליצור – אבל מה בעצם **נבנה** שם? נבין מה היא מערכת/אפליקציה, איך "מסתכלים" עליה ומפרקים אותה, ואיך AI עוזר להבין אותה לעומק.

יהודית ברדוגו

מערכות מותאמות אישית · נבנות בדיוק

יב

# לבנות זה לא מספיק – צריך להבין

גם כש-AI כותב את הקוד, את זו שמובילה. כדי לשלוט במערכת – צריך לדעת לקרוא אותה.

- כשמבינים את המבנה – אפשר **לבקש שינויים מדויקים** מ-AI, ולא רק "תתקני".
- כשמשהו **נשבר** – יודעים איפה לחפש ומה לשאול.
- אפשר **לקרוא קוד של אחרים** וללמוד ממנו, במקום לפחד ממנו.

🔗 המטרה במצגת הזו: להפוך את ה"קסם" למשהו **מובן** – לפרק את המערכת לחלקים שאפשר לתפוס.

# מה זה בכלל "מערכת" או אפליקציה?

לא קסם – אוסף מאורגן של הוראות שהמחשב קורא ומבצע.

כל אפליקציה – אתר, אפליקציית מובייל או מערכת ניהול – היא בסופו של דבר **אוסף קבצים** שמכילים **הוראות**. המחשב קורא את ההוראות לפי הסדר ומבצע אותן.

- ה"הוראות" כתובות בשפה שהמחשב מבין – זה **הקוד**.
- אותם קבצים יושבים בתיקיות, בדיוק כמו מסמכים במחשב.
- כשמשתמש לוחץ על כפתור – הוא בעצם מפעיל קטע הוראות שמישהו (או AI) כתב מראש.

 זה לא משנה אם הקוד נכתב ביד או ע"י AI – **התוצר זהה**: אותם קבצי הוראות.

# מה זה קוד? – הוראות בטקסט

דוגמה אמיתית: מה קורה כשלוחצים על כפתור "שמירה".

```
saveButton.onclick = () => {  
  const text = inputBox.value;  
  database.save(text);  
  alert("Saved!");  
};
```

"כשלוחצים על כפתור השמירה – בצע את הפעולות הבאות".

`saveButton.onclick`

קוראים את הטקסט שהשתמש הקליד בתיבה.

`inputBox.value`

שומרים את הטקסט במסד הנתונים (הזיכרון).

`database.save(text)`

מציגים למשתמש הודעה ש"נשמר".

`alert("Saved!")`

🔗 AI כמו Claude כותב בדיוק את הקוד הזה – באותה שפה ובאותו היגיון שמתכנת היה כותב.

# האנטומיה של כל מערכת – 3 שכבות

כמעט כל אפליקציה בנויה משלושת החלקים האלה.



## הזיכרון

Database

איפה המידע נשמר – משתמשים, הזמנות, תוכן.



## המנוע

Backend

הלוגיקה והחישובים שקורים מאחורי הקלעים.



## הצד שרואים

Frontend

הדפים, הכפתורים והעיצוב – מה שהמשתמש פוגש.

אנלוגיית מסעדה: ה-**Frontend** הוא האולם והתפריט, ה-**Backend** הוא המטבח שמכין, וה-**Database** הוא המזווה שבו שמורים החומרים.

# Frontend – הפנים של המערכת

כל מה שהמשתמש רואה ונוגע בו: דפים, כפתורים, טפסים, תמונות וצבעים. בנוי בעיקר משלוש שפות שעובדות יחד:

HTML השלד – מה יש בדף (כותרת, כפתור, תמונה).

CSS העיצוב – איך זה נראה (צבעים, גדלים, מיקום).

JavaScript ההתנהגות – מה קורה כשלוחצים ומקלידים.

👁️ טיפ: ברוב הדפדפנים, לחיצה ימנית ← "Inspect" מראה את ה-Frontend של כל אתר – דרך מצוינת ללמוד.

# המנוע והזיכרון – מה שלא רואים

## Database – הזיכרון

מקום קבוע לשמירת מידע:

- משתמשים, סיסמאות, פרופילים.
- תוכן, הזמנות, היסטוריה.
- נשאר גם אחרי שסוגרים את האתר.

## Backend – המנוע

הלוגיקה שרצה על שרת:

- בודק הרשאות (מי מותר לו לראות מה).
- מבצע חישובים וכללים עסקיים.
- מקבל בקשות ומחזיר תשובות.

 מה שמחבר בין הצדדים נקרא **API** – צינור מוסכם שדרכו ה-Frontend "מבקש" מידע מה-Backend ומקבל תשובה.

# איך נראים הקבצים של מערכת?

פרויקט הוא תיקייה עם קבצים מסודרים. כך נראה אחד פשוט:

`index.html` הדף הראשי – נקודת הכניסה (entry point) שנפתחת ראשונה.

`style.css` קובץ העיצוב של כל הדפים.

`app.js` הלוגיקה – מה קורה כשמשתמשים באתר.

`/components` תיקייה עם חלקים חוזרים (כפתור, תפריט, כרטיס).

`README.md` הסבר על הפרויקט – תמיד הקובץ הראשון לקרוא.

📖 השמות והתיקיות הם המפה. לא צריך להבין כל קובץ מיד – מתחילים מ-**README** ומנקודת הכניסה.



# איך "מסתכלים" על מערכת – מבחוץ פנימה

לא קוראים הכל מההתחלה. עוקבים אחרי המסלול של פעולה אחת.



## 4 · המידע

מאיפה נקרא / נשמר?



## 3 · הלוגיקה

מה ה-Backend מחשב?



## 2 · הפעולה

איזה קוד רץ כשלוחצים?



## 1 · המסך

מה המשתמש רואה ולוחץ?

בחרי פעולה אחת (למשל "התחברות") ועקבי אחריה לאורך כל המסלול – כך מבינים מערכת שלמה, חתיכה-חתיכה.

# שאלות שכדאי לשאול על כל מערכת

- מה היא עושה? – מה הבעיה שהמערכת פותרת, במשפט אחד.
- מי המשתמש? – למי זה מיועד ומה הוא רוצה להשיג.
- מה קורה כשלוחצים X? – לבחור פעולה מרכזית ולעקוב אחריה.
- איפה המידע נשמר? – איזה נתונים יש, ואיפה הם יושבים.
- מה מתחבר למה? – אילו חלקים מדברים זה עם זה (Frontend ↔ Backend ↔ DB).

☑ חמש השאלות האלה נותנות תמונה מלאה של כל מערכת – עוד לפני שקראת שורת קוד אחת.

# להשתמש ב-AI כדי להבין לעומק

Claude Code לא רק כותב – הוא מסביר. תני לו את הפרויקט ושאלי.

"תסביר לי בעברית פשוטה מה הקובץ הזה עושה"



"תן לי סיכום של מבנה הפרויקט – מה כל תיקייה"



"איפה בקוד מטופלת ההתחברות של משתמש?"



"תאר את הזרימה מרגע הלחיצה ועד שהמידע נשמר"



אפשר תמיד לבקש "תסביר כאילו אני מתחילה". זה הכוח האמיתי: ללמוד את המערכת תוך כדי שעובדים עליה.



# איך לחקור פרויקט חדש – צעד-צעד

- 1. קראי קודם את ה-**README** – הוא מספר מה הפרויקט עושה ואיך מריצים.
- 2. בקשי מ-**Claude Code** "מפה" של הפרויקט ושל נקודת הכניסה.
- 3. בחרי **פעולה אחת** ועקבי אחריה מבחוץ פנימה (שקף 9).
- 4. שני **דבר קטן** (טקסט, צבע) – והריצי כדי לראות מה השתנה.
- 5. טעית? **חזרי אחורה עם Git** – ככה לומדים בלי פחד.

הדרך הכי טובה להבין מערכת היא **לגעת בה**: לשנות, להריץ, ולראות מה קרה.

# מערכת היא לא קסם – היא הוראות שאפשר לקרוא

Database, Frontend, Backend · מסתכלים מבחוץ פנימה · שואלים את חמש השאלות · ונעזרים ב-AI כמורה פרטי. ככה הופכים כל מערכת ממפחידה – למובנת.